

Multi-Channel Hebbian Plasticity in Multilayer BDH Solves Context-Conditional Binding by Partitioning Memory

An early-training recipe finds the partition without restarts in most runs for two streams
and for four with spare channels; at eight streams the gate forgets the context

A technical report • Revision 7 (preliminary)

Project repository: [Ynkling/clankers](#)

Abstract

Revision 6 showed, on a context-conditional binding task, that two Hebbian memory channels with a hand-set routing bind two conflicting streams where one channel cannot, that a gate trained from the task loss alone finds that routing on about half of its runs, and that a label-free restart rule made two streams reliable. Four streams were the open problem. This revision adds Phase V: five pre-registered tests, each run on two machines, and thirteen batches of exploratory screens.

The central result is an early-training recipe that finds the partition without restarts. For the first 2400 updates the memory trains at a tenth of the gate’s learning rate, and a hinge penalizes the gate when its channel choice at key positions is explained by position. At two streams the recipe discovered the routing on 19/20 and 20/20 fresh seeds on the two machines, against 8/20 on each for the plain gate. Slowing the memory alone beat the plain gate on both machines, and dropping the slow phase weakened the hinge on both; the hinge’s gain over slow memory alone was shown on one machine (4 vs 0, $p = 0.0625$, on the other). The recipe carried to eight keys (20/20, 17/20, beating the plain gate on both machines). At four streams with sixteen channels it bound 17/20 and 18/20 without restarts, beating the plain gate on L but not on X (7 vs 2, $p = 0.09$). The hinge fires rarely and early, and in screens each firing was a push on the gate hundreds of times larger than the task gradient; switched off after update 2400 it lost one of 80 screened runs. Under the Muon optimizer the recipe also works, and binds four streams sooner. A pre-registered test showed both on one machine; on the other, with the same seeds and the four-stream part cut to 12 of them, both effects pointed the same way without reaching significance.

Eight streams remain unsolved. No learned-gate run trained on all eight streams from the start has bound, and a curriculum over the number of streams reaches at most 15/36, mostly without one channel per stream. Screens trace the failure to the gate. Its recurrent state carries the stream at initialization (median decodability 0.91, chance 0.125) and loses it within the first 600–1200 updates, as its recurrent term grows from 0.04 to 5–7 by update 2400 while its input term stays between 0.06 and 0.34. In a later screen the stream was lost, within 50 updates, in each of the seven of ten runs where the spectral radius of the gate’s recurrent weights rose clearly above one. A cap on that recurrence is being screened. We also correct several of our own errors, among them pooled cross-machine p -values in Revision 6 that treated the two machines as independent although they ran the same seeds.

Revision 7 (preliminary). This revision adds Phase V, the five pre-registered tests and thirteen exploratory batches run since Revision 6 (Table 1), and condenses Phase IV into Section 2; Revision 6 has its full account. Two machines ran every Phase V test: **X**, a cloud container, and **L**, an Intel i7-12650H. X’s container ran on two Intel Xeon hosts, at 2.10 and 2.80 GHz; Adam runs reproduce bit for bit across them, and X’s results recorded after the stream-recipe test come from the 2.80 GHz

host. A third container, **E**, runs the exploratory screens on a separate branch (Section 6); its batches 2–14 ran on a 2.10 GHz Xeon, apart from two segments of batch 7. Every verdict is per machine. Both machines run the same seeds, so pooled counts are descriptive and carry no p -value (Section 15). L’s results for four of the five tests are not yet committed to the repository; we regenerated their verdicts from L’s results files with each test’s own report function (Appendix A). The revision is preliminary: batch 15 is still running (its spectrum screen has finished, and we read that screen’s saved records ourselves, before E’s report), the screens are screens, and everything is one small model on a synthetic task.

test	question	verdict on X	verdict on L	§
stream_recipe	four streams: sixteen channels plus an early check and restarts	R1 MAJORITY; R2 NOT PRECISE; R3 SHOWN; R4 NOT SHOWN	R1 RELIABLE; R2 PRECISE; R3 SHOWN; R4 NOT SHOWN	4
stream_curriculum	eight streams by a $2 \rightarrow 4 \rightarrow 8$ stream curriculum	VALID; C1 NOT SHOWN; C2 MINORITY; C3 UNTESTABLE	same	5
slow_start	slow memory plus a hinge, no restarts	H0, HA, S0 SHOWN; HB, H0S NOT SHOWN	H0, HA, HB, S0, H0S SHOWN	7
recipe_scope	does the hinge need the slow phase; the recipe on the curriculum	Q2 SHOWN; Q1, Q3, Q4, Q5 NOT SHOWN	Q2, Q3 SHOWN; Q1, Q4, Q5 NOT SHOWN	8
early_recipe	the hinge in updates 1–2400 only; Muon	E1, E2 NOT SHOWN	E1, E2 SHOWN	9.2
exploratory batches 2–14 (E)	34 screens of outside ideas and diagnostics	see Sections 6, 9–12		
batch 15 (E)	the gate’s recurrence at eight streams: a cap (S41), its spectrum (S42)	S41 running; S42 finished		11

Table 1: Phase V. Each test is a file `test_<name>.py` whose docstring records its pre-registered design and, after the run, its result. Every verdict was fixed before the test ran and printed mechanically. Interpretation beyond the verdict is labelled as diagnostic or post hoc wherever it appears. Appendix A lists the commits.

1 Background

BDH [1] stores working memory in synaptic state updated by a Hebbian rule. In its GPU form the operation is linear attention: the score between a query position t and a past position s is a content match $x_t \cdot x_s$ between sparse positive neuron activations, weighted by a positional operator (scalar decay or RoPE). The public implementation, `bdh.py`, ties queries to keys ($Q = K$), shares weights across layers, and LayerNorms the attention output.

The extension gives every synapse k channels and every token a gate $g_t \in \Delta^{k-1}$ that distributes its write over the channels and blends its read from them. With one distribution for both, the effective score becomes

$$\underbrace{(x_t \cdot x_s)}_{\text{content match}} \times \underbrace{(g_t \cdot g_s)}_{\text{context-gate match}}, \quad (1)$$

so tokens sharing a context can read each other’s memory while tokens in different contexts are isolated. With $k = 1$ the model is single-channel BDH. The learned gate (arm A) is a small

recurrence over the token embeddings v_t ,

$$h_t = \tanh(W_{\text{in}} v_t + W_h h_{t-1}), \quad g_t = \text{softmax}(W_g h_t), \quad (2)$$

with 32 units in h_t . We call $W_{\text{in}} v_t$ the gate’s *input term* and $W_h h_{t-1}$ its *recurrent term*. The hand-set *perfect gate* sends every token of stream s to channel s .

The uniform gate is a stationary point. Writing each gate as a deviation from uniform, $g_t = \frac{1}{k} \mathbf{1} + \delta_t$ with $\sum_c \delta_t[c] = 0$,

$$g_t \cdot g_s = \frac{1}{k} + \delta_t \cdot \delta_s. \quad (3)$$

The constant is cancelled by the LayerNorm after attention, so routing enters only through products of deviations, and at the uniform gate the task loss exerts no first-order pull toward any routing. The recurrent gate starts almost exactly uniform; its gradient norm at step 1 is below 10^{-5} , against about 2.6 for the rest of the model. Because the gate’s output passes through a softmax, the k rows of the gradient into W_g add up to zero, so its rank is at most $k - 1$.

A short causal convolution. Since Phase IV most arms include the short convolution that Mamba and most modern linear-attention models place before their token mixer [2]: a depthwise causal filter of width 4 at every layer, initialized to the identity. It feeds the queries and the values; the residual stream and the gate’s input are not convolved.

Two optimizers. All main-line tests before Phase V used Adam. Phase V adds Muon [3, 4], which replaces each 2-D weight’s momentum by an approximately orthogonalized version (five Newton–Schulz steps), so the size of an update does not depend on the size of the gradient. We use it on every 2-D weight and on each head slice of a 3-D weight, with Adam on the embedding, the convolution and any 1-D parameters.

2 Earlier phases in brief

Phases I–III (Revisions 1–5). A gate that was a function of token identity could not express the routing (Phase I). A recurrent gate could, and separated two streams (Phase II). On the binding task of Section 3, two channels with the perfect gate bind two streams where no single-channel model binds, and a label-free learned gate finds the routing on about half its runs, beating every single-channel reference on both machines (Phase III).

Phase IV (Revision 6). Ten pre-registered tests, each on both machines. A readout from the gate’s state into the residual stream harms the gate through its gradient. At two streams, more channels did not raise the gate’s rate, but a restart rule that reads only held-out accuracy at step 2400 succeeded in 60/60 trials. A short convolution does not replace the gate at the working learning rate 10^{-3} (one channel 7/80, gate 51/80). At 4×10^{-3} , though, one channel with the convolution binds 34/80, by carrying the context to the value position through its lag-2 weight. With eight keys per stream the gate binds from scratch in 49/70 runs at 10^{-3} , while one channel with the same fast-weight memory and 1.8 times the parameters binds 0/24: the advantage is the partition, not the memory. With four streams and four channels the gate bound 14/40, mostly failing by merging streams into a shared channel; eight or sixteen channels removed most merges. What remained was an early commitment to a split by something other than the stream.

3 The task, the outcomes and the recipes

Task. There are S streams, each with a context token CTX_s . For each of P keys, S distinct values are drawn from 16 value tokens, so every key is bound to a different value in every stream. In the *grouped* layout the body is SP triples $[\text{CTX}_s, \text{KEY}_i, \text{VAL}_{i,s}]$, grouped by key, with keys in random order and stream order random within each key’s group; every key is immediately preceded by its own stream’s token. One query $[\text{CTX}_q, \text{KEY}_q, ?]$ ends the sequence. Chance among values is $1/16$; a model that binds keys to values but cannot tell streams apart scores $1/S$. The *header* layout, used only in screens, gives each stream one block that starts with its context token, so most items are far from it. The model has three layers, decay 0.95, $N = 256$, embedding width 32 and one head, and trains at batch 32 for at most a budget set per test, usually 24,000 or 28,800 steps, with evaluation every 1200 steps on 2048 held-out queries; a run stops early after three consecutive evaluations at or above 0.95.

Outcomes.

- **Bound:** an evaluation reaches 0.95 and every later one stays there; the *transition* is the first such step.
- **Discovered** ($k = S = 2$): bound, with the streams’ read gates separated at value positions.
- **Bound routed** ($k \geq S > 2$): bound, every stream has its own channel (a one-to-one *stream-to-channel map*), and every stream’s accuracy is at least 0.9. At $S = 4$, $k = 4$ the memory occasionally binds without the partition (one screened run bound with all four streams on one channel), so routing, not binding, is the outcome that counts there.
- **Failure classes:** POSITION or KEY when the gate’s channel choice at key positions is explained ($\eta^2 \geq 0.5$) by position (triple index or sequence half) or by key identity; STREAM-PARTIAL; OTHER. For $S > 2$: MERGED (j share) when j streams share a channel, non-stream when the gate does not split by stream, *collapsed* when final accuracy is below 0.15.
- **ROUTED*** (screens): routing margin at least 0.9 and the gate’s η^2 by stream above 0.9, measured at a given step.

Recipes.

- **SLOW:** one Adam optimizer with two groups. The gate (W_{in}, W_h, W_g) trains at 10^{-3} throughout; every other parameter, embedding and convolution included, at 10^{-4} for updates 1–2400 and 10^{-3} after.
- **HINGE:** SLOW plus $1.0 \times [\text{relu}(\eta_{\text{index}}^2 - 0.2) + \text{relu}(\eta_{\text{half}}^2 - 0.2)]$, where the η^2 terms measure how much of the read gate’s variance at key positions is explained by triple index and by sequence half, on each training batch, pooled over channels. Its gradient reaches the gate and the embedding. It uses no stream labels, but it needs to know where the key positions are. We say the hinge *fires* on a batch when a term is above zero.
- **WINDOW:** the hinge with weight 1 on updates 1–2400 and 0 after.
- **MUON:** Muon at learning rate 0.005 on every 2-D weight (momentum 0.95, Nesterov); the gate group at 0.005 throughout, the other Muon weights at a tenth of that and the Adam groups at 10^{-4} for updates 1–2400, then the full rate. Of the screened rates 0.005, 0.01 and

0.02, it was the smallest, and the only one at which the perfect gate bound on both screening seeds.

Statistics. Paired comparisons use exact McNemar tests, unpaired ones Fisher’s exact test; “ b vs c ” gives the discordant pairs. Tests reuse seeds on purpose, so that a new arm pairs with a run recorded earlier on the same machine. Every such test first reproduces a recorded run exactly before using the file.

4 Four streams: spare channels and an early check

Revision 6 found, post hoc, that at four streams with spare channels a held-out accuracy of at least 0.4 at step 4800 picked binders with 38/38 precision. The stream-recipe test put that into a restart rule on fresh seeds 240–259 ($S = 4$, $P = 4$, convolution, Adam 10^{-3} , up to five attempts per trial), with sixteen channels and with four as a control.

bound, of 20	X	L	both
A4k16, $k = 16$, plain	12	9	21/40
A4k16_R, $k = 16$, restart rule	17	19	36/40
A4k4, $k = 4$, plain	10	5	15/40
A4k4_R, $k = 4$, restart rule	10	8	18/40
perfect gate, $k = 16$ (of 3)	3	3	6/6

Table 2: The stream-recipe test. R1 (the recipe is reliable): MAJORITY on X (17/20), RELIABLE on L (19/20). R2 (the check is precise, at least 0.9 of passing attempts bind): NOT PRECISE on X (17/20, 0.85), PRECISE on L (19/19). R3 (spare channels make restarts work, A4k16_R vs A4k4_R): SHOWN on both (Fisher $p = 0.020$ on X, 2×10^{-4} on L). R4 (spare channels alone, on fresh seeds): NOT SHOWN on both (5 vs 3, $p = 0.36$; 8 vs 4, $p = 0.19$).

The check works only with spare channels. At $k = 4$ every attempt that passed the check and then failed was a merge (9 of 9 on X, 12 of 12 on L), because a run with two streams sharing a channel already reaches about 0.75, and one with three about 0.5. On X the three passing attempts that failed at $k = 16$ were also merges. With restarts, four streams bind on 36 of 40 runs. Part 2 looked at eight streams descriptively on L: the perfect gate bound 2/2, and the restart arm 0/5, with all 25 attempts at 0.06–0.09 at step 4800 (X dropped Part 2 under its time rule).

5 Eight streams by a stream curriculum

From scratch, eight streams ($S = 8$, $P = 4$, $k = 16$, convolution, 99 tokens) did not bind. The stream-curriculum test trained on 2 of the 8 streams per sequence for updates 1–4800, on 4 for 4801–9600, then on all 8, at 10^{-3} with one optimizer throughout. Arms: SC8 (curriculum), SC8_R (curriculum with a restart rule on two-stream accuracy at step 2400), D8 (all eight from step 1), and a perfect-gate validity arm. On X a time rule cut D8 to seeds 260–265 and the curriculum arms to 260–275; L ran 260–269 and 260–279.

Both machines were VALID (the perfect gate bound 3/3 on each). C1 (the curriculum beats training from scratch) was NOT SHOWN on either: SC8 bound 6/16 on X and 4/20 on L, D8 0/6 and 0/10 (3 vs 0, $p = 0.125$; 2 vs 0, $p = 0.25$). C2 was MINORITY on both (SC8_R 7/16, 5/20), and C3 UNTESTABLE on both, because no run was routed at step 4800. No D8 run bound: all of X’s six and eight of L’s ten ended below 0.15, and L’s other two near 0.2, with streams merged four

or three to a channel. On X every curriculum binder bound at step 10,800 or 12,000, soon after the switch to eight streams, and three of its binders had no one-to-one map. Pooled over both machines (descriptively), SC8’s failures were 8 collapses onto one channel, all position splits, and 18 merges.

6 Screens: what decides the gate’s first move

A separate session screens ideas on its own branch, writes only new files, and labels every output “exploratory, not a result”. A promising screen returns to the main line for a pre-registered test. Most screens in batches 2–9 asked how to move the gate’s first commitment toward the stream at two streams, four keys, $k = 2$, without the convolution, on seeds 160–199 (Table 3); others looked at distant cues (Section 12) and at merges (Section 10). Each arm is paired by seed with X’s recorded arm A, or with an earlier screen. A typical rule, fixed before the run, called a screen promising if it won at least 4 more discordant pairs than its comparison with McNemar $p < 0.1$.

screen	change	discovered	comparison	verdict
A (X, recorded)	—	12/40	—	—
kWTA warm-up (S4)	sparse codes in the memory for 2400 updates	7/20	A 7/20	not
slow memory (S5, S7)	SLOW	24/40	A 12/40; 15 vs 3	— ^a
gate reset (S11)	reset the gate if position or key explains it at 1200	10/20	A 7/20; 3 vs 0	inconclusive
position penalty (S12)	SLOW + a linear penalty on η^2 by position	15/20	A 7/20; 9 vs 1	promising
hinge (S13)	HINGE	34/40	SLOW 24/40; 10 vs 0	promising
random kick (S16)	a random push of the hinge’s size, on the batches where it fires	12/20	HINGE 14/20; 1 vs 3	“suffices” ^b
untimed kick (S20)	one random push at update 120	27/40	HINGE 34/40; 0 vs 7	inconclusive
gate noise (S26)	SLOW + decaying Gaussian noise on the gate	27/40	HINGE 34/40; 0 vs 7	inconclusive

Table 3: Two-stream screens on seeds 160–199 (Adam, 10^{-3} , no convolution); 20-seed screens use 160–179. ^aS5 on 160–179 was inconclusive, S7 on 180–199 promising; pooled here, with no verdict on the pool. ^bRun on the 20 seeds where the hinge fired; “a kick of that size suffices” by its rule ($\geq 12/20$), on the boundary. The untimed kick and the gate noise were each compared with SLOW by their rules (27 vs 24 of 40, inconclusive) and with HINGE as printed.

Two observations from batches 1–4 shaped the rest. First, routing at step 1200 predicts routing at the end (first seen in batch 1), though not binding. Sparse codes in the memory made the gate route early (ROUTED* at 1200 in 15/20 against 6/20) without making it bind, and slowing the memory moved the first commitment toward the stream (23/40 against 11/40 routed at 1200). We read early training as a race between the gate finding the stream split and the memory exploiting a non-stream one. Second, slowing the memory left the gate’s failures as position splits (11 of 16 failures over seeds 160–199). The hinge targets exactly those, and it removed them: on the same seeds it discovered on 10 seeds where slow memory alone did not, and on none where it lost.

The hinge acts rarely. It never fired on 20 of 40 seeds, whose runs are bit-identical to slow memory alone; where it fired, it usually did so on 1–3 of the roughly 5000 batches before binding. A random push of the same size on the same batches did most of what the hinge does (12/20 against 14/20), and a single random push at a fixed update did not (27/40 against 34/40, 0 vs 7). Timing appears to do most of the work; whether the hinge’s direction adds anything was not resolved (the random push against the hinge was 1 vs 3, $p = 0.63$; Section 9).

7 Slow memory and a hinge, confirmed

The slow-start test confirmed the recipe on fresh seeds and carried it to the working configurations, with no restarts. Part 0 ran arm A, SLOW and HINGE at two streams, in the screens’ configuration on fresh seeds 280–299. Parts A and B added the hinge to recorded runs: eight keys (DIRECT8, $S = 2$, $P = 8$, convolution, seeds 220–239), and four streams with sixteen channels (A4k16, seeds 240–259). Part C looked at eight streams descriptively.

part	arm	X	L	both
0: $S=2$, $P=4$, discovered	A0, arm A	8/20	8/20	16/40
	SLOW0	15/20	15/20	30/40
	HINGE0	19/20	20/20	39/40
A: $S=2$, $P=8$, bound	DIRECT8 (recorded)	13/20	12/20	25/40
	HINGE8	20/20	17/20	37/40
B: $S=4$, $P=4$, $k=16$, bound	A4k16 (recorded)	12/20	9/20	21/40
	HINGE4k16	17/20	18/20	35/40
C: $S=8$, $k=16$, bound	D8 (recorded)	0/6	0/10	0/16
	HINGE_D8	0/10	0/10	0/20

Table 4: The slow-start test (no restarts). On X: H0 (HINGE0 beats A0) SHOWN, 11 vs 0, $p = 5 \times 10^{-4}$; HA (HINGE8 beats DIRECT8) SHOWN, 7 vs 0, $p = 0.008$; HB (HINGE4k16 beats A4k16) NOT SHOWN, 7 vs 2, $p = 0.09$; S0 (SLOW0 beats A0) SHOWN, 9 vs 2, $p = 0.033$; H0S (HINGE0 beats SLOW0) NOT SHOWN, 4 vs 0, $p = 0.0625$. On L all five SHOWN: 12 vs 0, $p = 2 \times 10^{-4}$; 5 vs 0, $p = 0.031$; 11 vs 2, $p = 0.011$; 10 vs 3, $p = 0.046$; 5 vs 0, $p = 0.031$. Bands: HINGE0 RELIABLE on both; HINGE8 RELIABLE on X, MAJORITY on L; HINGE4k16 MAJORITY on X, RELIABLE on L. Part C: 4 of 10 HINGE_D8 runs collapsed on each machine, so its reading was “it does not” (prevent the collapse).

The readings were “the screen replicates on fresh seeds” on both machines. The second reading was “it carries to eight keys only” on X and “it carries to the working configuration” on L. Over both machines a single HINGE0 run failed (X’s seed 287, an OTHER failure on which the hinge fired 950 times). Plain arm A failed mostly by position splits (9 of 12 failures on X, 8 of 12 on L), and SLOW alone almost only by position splits (4 of 5 on X, the fifth OTHER; 5 of 5 on L). The hinge fired rarely here too: it never fired on 11 of X’s 20 HINGE0 runs (9 of L’s), and on X no run saw more than two firings in the first 2400 updates. At four streams HINGE4k16 had no non-stream failures on X, where A4k16 had four; it lost two of A4k16’s late binders, and HB missed on X by 7 vs 2. At eight streams no HINGE_D8 run bound and four of ten collapsed on each machine, although six of ten on each ended above 0.15 (up to 0.47 on X and 0.50 on L), where X’s D8 runs ended at 0.08–0.10.

8 Which part of the recipe matters

The recipe-scope test asked whether the hinge needs the slow phase (HONLY: the hinge with one learning rate throughout), and whether the recipe helps the stream curriculum (SC8_H: the stream curriculum with HINGE, the hinge reading each stage’s key positions).

At two streams the hinge without the slow phase removed the position splits, but key splits took their place: HONLY0’s failures were 5 KEY of 8 on X and 5 of 7 on L. Slowing the memory prevents those. At sixteen channels the hinge alone did about as well as the full recipe (Q4), and on neither machine did a hinge-only run fail by a key or position split. The slow phase may matter only when

	X	L
HONLY0, $S=2$, discovered (A0; HINGE0)	12/20 (8; 19)	13/20 (8; 20)
HONLY4k16, $S=4$, $k=16$, bound (A4k16; HINGE4k16)	16/20 (12; 17)	18/20 (9; 18)
SC8_H, $S=8$ curriculum, bound (SC8)	8/16 (6)	7/20 (4)

Table 5: The recipe-scope test, with the recorded runs on the same seeds in parentheses. Q1 (HONLY0 beats A0) NOT SHOWN on both (6 vs 2, $p = 0.14$; 6 vs 1, $p = 0.0625$). Q2 (HINGE0 beats HONLY0) SHOWN on both (8 vs 1, $p = 0.020$; 7 vs 0, $p = 0.008$). Q3 (HONLY4k16 beats A4k16) NOT SHOWN on X (6 vs 2, $p = 0.14$), SHOWN on L (9 vs 0, $p = 0.002$). Q4 (HINGE4k16 beats HONLY4k16) NOT SHOWN on both (4 vs 3; 2 vs 2). Q5 (SC8_H beats SC8) NOT SHOWN on both (6 vs 4, $p = 0.38$; 6 vs 3, $p = 0.25$). SC8_H band: MAJORITY on X, MINORITY on L. Reading on both: “both parts are needed”.

channels are few; this is an observation, not a tested claim. On the eight-stream curriculum the hinge removed the collapses (X: 0 against SC8’s 4; L: 1 against 4), but merges took their place, and most binders bound without one channel per stream (five of X’s eight, six of L’s seven). On X, nine of 16 SC8_H gates held the eight streams in two channels of four at step 4800.

9 The hinge is an early kick; Muon

9.1 Screens

The screens then asked what the hinge does, and whether the recipe survives a different optimizer.

- **Muon** (S25, seeds 160–179). Muon with SLOW and the hinge discovered 20/20, all routed at step 1200. Without SLOW and without the hinge it discovered 9/20, and all 11 failures were position splits.
- **The hinge under Muon** (S32, 160–199). Without the hinge, Muon with SLOW discovered 31/40; with it, 39/40 (8 vs 0, $p = 0.008$). On every rescued seed the hinge first fired between updates 50 and 168. In both arms routing appeared between updates 600 and 900 (ROUTED* at 900: 38/40 with the hinge, 27/40 without).
- **An early window** (S34, 160–199). With the hinge on updates 1–2400 only, Muon discovered 38/40 against 39/40 for the full hinge (1 vs 0) and Adam 34/40 against 34/40 (0 vs 0). The one loss, a Muon seed, had needed 277 late firings.
- **Four streams under Muon** (S35, 240–259, $k = 16$). Muon with HINGE bound 17/20, the same as X’s HINGE4k16, with 15 runs bound by update 2400. X’s transitions ranged from 3600 to 21,600.

What a firing does. At a firing, the hinge’s gradient on the gate was a median 850 times the task gradient under Muon and 726 times under Adam (S34). Under Muon the gate’s momentum stayed aligned with that push (cosine at least 0.5) for a median of 59 updates; because Muon’s step size does not depend on the gradient’s size, the gate moves in the hinge’s direction for that stretch. Under Adam a gradient far above its running average gives a sign-like full step. Either way a firing is a large, directional push on the gate, delivered early: the first firings came at updates 50–168, and no run was routed at update 300 or 600 (S32), so the gate was presumably still near the uniform stationary point of equation (3). A batch 7 screen had measured Adam’s second moment for W_g rising by a factor of 10^7 or more at a firing, and we had suggested that this then froze the gate. Muon has no second moment and the hinge still works, so the freeze is not needed.

9.2 The early-recipe test

The early-recipe test tested two screened findings on fresh seeds. Part A ($S = 2$, seeds 300–339) compared MUON alone (SLOW_M) with MUON and WINDOW (WIN_M). Part B ($S = 4$, $k = 16$, seeds 340–359) compared WINDOW under Adam (WIN16_A) and under Muon (WIN16_M). On X a time rule cut Part B to seeds 340–351.

	X	L
SLOW_M, discovered	36/40	34/40
WIN_M, discovered	40/40	40/40
WIN16_A, bound (by update 4800)	10/12 (6)	17/20 (12)
WIN16_M, bound (by update 4800)	11/12 (10)	19/20 (19)
perfect gate under Muon, Parts A and B	2/2, 2/2	2/2, 2/2

Table 6: The early-recipe test. E1 (WIN_M beats SLOW_M): NOT SHOWN on X (4 vs 0, $p = 0.0625$), SHOWN on L (6 vs 0, $p = 0.016$). E2 (WIN16_M bound by update 4800 more often than WIN16_A): NOT SHOWN on X (5 vs 1, $p = 0.11$), SHOWN on L (7 vs 0, $p = 0.008$). Bands: WIN_M RELIABLE on both; WIN16_M RELIABLE on both; WIN16_A MAJORITY on both. Readings on L: “the early hinge window works under Muon” and “Muon binds four streams sooner”.

On X all four SLOW_M failures were position splits, and WIN_M rescued them with firings between updates 88 and 230; on L it rescued all six, five position splits and one STREAM-PARTIAL. Under Muon the four-stream binders bound at a median of update 2400 on both machines, against 4800 under Adam, and by step 4800 the four streams sat on four distinct channels in 11 of 12 runs on X and 19 of 20 on L (Adam: 6 of 12, 12 of 20).

The two machines are not independent replicates. They ran the same seeds, so each pair of runs started from the same weights and saw the same batches, and seed-level outcomes largely agree (38/40 for SLOW_M, 40/40 for WIN_M, 9 of 12 and 10 of 12 for the Part B arms). Their numerics differ, though: no run reproduced across the two machines in any arm, the Adam-only arm included, and Muon’s bf16 Newton–Schulz step differs even between X’s two Xeon hosts. What the test establishes is one pre-registered success on L, and the same direction on X with no reversals in E1. Two things weakened X’s run. The screens had shown 31/40 without the hinge, while the fresh seeds gave SLOW_M 36/40 on X (34/40 on L), leaving less room. And the time rule cut Part B to 12 seeds.

10 Merges with four channels

At four streams with four channels the failures are merges, and spare channels are the main fix (Section 4; descriptively, plain runs merged in 32/80 at $k = 4$ against 9/80 at $k = 16$ over the two tests). The screens asked whether a merge, once formed, can be broken, using X’s eight recorded four-channel runs that were merged at update 9600 and still merged at the end, and continuing each for 9600 updates.

- Repeated large pushes on the gate (S21): 0/8 split. A router z-loss [5] (S27): 2/8. The z-loss did lower the logits’ scale, but by lowering all logits together, along the one direction the softmax ignores.
- Copying the busiest channel’s gate row onto the idle one, with a little noise and a reset of that weight’s optimizer state (S24): 8/8 split, against 1/8 for the noise and reset alone. The

merged streams’ gate states were nearly identical, yet the copy, which puts the idle channel level with the shared one, let small differences grow.

- A label-free trigger for that copy (S36, from scratch, Muon, $k = 4$): bound 5/10 against 4/10. The trigger split when probe accuracy stalled below 0.95, aiming at the channels with the most and least read-gate mass over all positions. That aim was wrong in about half of its 31 splits, because a channel that holds no stream at key positions can carry the most mass elsewhere. In three runs held-out accuracy fell from about 0.75 to 0.35–0.51 after a split, before returning to the plateau.
- Aiming by the mass at key positions instead (S37): under Muon 10 of 11 splits were on target, against 5 of 11 by the all-position rule; under Adam the two rules aimed about equally well. Bound with one channel per stream: Muon 6/10 against 3/10 without splitting; Adam 6/10 against 5/10.

Muon on its own did not break merges: at $k = 4$ it bound 3/10 and 4/10 from scratch (S33a), and of the 15 runs merged at update 4800, 14 were still merged at the end. Aiming is not the whole problem: in the four Muon runs that stayed merged under S37, both splits were on target and the stream-to-channel map did not change.

11 Eight streams: the gate forgets the context

Every learned-gate arm trained on all eight streams from step 1 has failed, under Adam and Muon, with and without the hinge, with plateau-triggered splits, and with restarts. That is none of more than 100 runs, while the perfect gate binds. Four screens located the failure, and a fifth checked our reading of it.

- **Splitting does not help** (S36b, 43,200 updates): 0/10 under each optimizer. Every run split 7–12 times, but a split’s new channel rarely held a stream by the next check.
- **The gate’s state does not carry the stream** (S38). A nearest-class-mean decoder of stream identity read the stream from h at key positions at a median of 0.13 (Muon) and 0.14 (Adam) at update 4800, against a chance level of 0.125. At four streams it read 0.84 ($k = 4$) and 1.00 ($k = 16$). Since each key’s own stream token sits one position earlier, the gate needs only one token of memory.
- **The stream is there at the start and is lost** (S39, Table 7). At initialization the decoder reads 0.91: with small initial weights, h at a key still carries the stream token one step back. Training removes this memory. The loss coincides with the recurrent term overtaking the input term, by update 400 under Muon (0.20 against 0.10) and 600 under Adam (0.14 against 0.08), and then growing past 1.9 (Muon, update 600) and 3.7 (Adam, update 1600). By update 2400 the state at a stream token no longer decodes that token either. This is mostly not tanh saturation: at update 2400 the median share of saturated units is at most 15% (one run reached 57%). The hinge only brings the loss forward slightly. At four streams the input term at stream tokens grows instead (from 0.06 to 0.55), and the memory survives.
- **Feeding the gate the previous token** (S40). Adding $W_{\text{prev}}v_{t-1}$ to the gate’s pre-activation gives it the stream directly: its input decodes the stream at 0.97–1.00 under Muon. Its state still reads only 0.53–0.64, diluted by the recurrent term. No run bound (0/10 under each optimizer). The failures shifted from non-stream splits to merges on three or four channels (9 of 10 under Muon), and final accuracy rose in 9 of 10 Muon pairs.

update	0	200	400	600	1200	2400
<i>Muon</i> , $S = 8$, $k = 16$, <i>HINGE</i> (seeds 260–264)						
decodability of the stream from h at keys	0.91	0.70	0.39	0.18	0.15	0.13
recurrent term $ W_h h_{t-1} $ at keys	0.04	0.08	0.20	1.91	4.02	6.93
input term $ W_{in} v_t $ at keys	0.06	0.07	0.10	0.11	0.15	0.13
<i>Adam</i> , $S = 8$, $k = 16$, <i>HINGE</i> (seeds 260–264)						
decodability	0.91	0.88	0.79	0.50	0.24	0.14
recurrent term	0.04	0.05	0.06	0.14	0.32	5.08
<i>Muon</i> , $S = 4$, $k = 4$, <i>HINGE</i> (seeds 240–244; chance 0.25)						
decodability	0.95	0.84	0.62	0.61	0.85	0.83
recurrent term	0.04	0.10	0.49	0.58	1.67	2.27

Table 7: Screen S39: medians over five reruns, each reproducing its recorded curve. Chance is 0.125 at eight streams. Under Muon without the hinge the decodability is the same through update 200, higher at 400 and 600 (0.51, 0.26) and the same from 1200; its recurrent term at 600 is 2.28.

The jump in the recurrent term between updates 400 and 600 under Muon is sudden. Our reading was that the recurrence’s gain crosses one: past that point the state sustains itself and ignores its input. At eight streams the gate’s output stays near uniform (entropy 2.70 of a possible 2.77 at update 2400 under Muon), so the task gives its input weights little reason to grow.

The gain crossing (S42, batch 15). This screen reran the two eight-stream configurations of Table 7 to update 1600 on seeds 260–264 and logged the spectral radius ρ of W_h every 50 updates. It ran on E’s new 2.80 GHz host, where Muon runs no longer reproduce S39’s records (Section 15), so its Muon runs are new trajectories on the same seeds; its Adam runs reproduce X’s records. At initialization ρ was 0.52–0.63.

- In seven of the ten runs ρ rose clearly above one (peaks 1.20–2.13), first crossing it between updates 350 and 1400. In each of them, the stream’s decodability from h at keys fell to 0.13–0.26 within 50 updates of the crossing, and the recurrent term passed 1 within 100 updates.
- In the other three (one Muon, two Adam) ρ peaked at 0.88–1.03, and the stream was only partly lost (0.31–0.72 at update 1600).
- In two Muon runs ρ later fell back below one, and decodability partly recovered (to 0.50 and 0.69).

The screen’s pre-set rule asks whether, in at least four of five runs, the recurrent term first exceeds 1 within 150 updates after ρ first does. It labels the Muon configuration “gain crossing” (4 of 5) and the Adam configuration “not” (3 of 5; in its two other runs ρ never exceeded one). We applied that rule to the saved records ourselves, before E’s report. The pattern is what the gain-crossing reading predicts, but it is a correlation in ten runs. The intervention is S41, still running: a cap of 0.5 on the spectral norm of W_h , the cap together with the previous-token input, and the previous-token input with no recurrence at all. Four of its 55 runs are saved, one seed per arm; none bound, and four runs cannot be read.

12 Distant cues

A language model’s context cues are usually far from the tokens they govern. The header layout puts each stream’s token once, at the start of its block. In screens:

- The plain gate discovered 0/10 there (S6). The perfect gate bound 3/3. Trained on the routing directly, the recurrent, fading-memory and selective gates could express it; the local gate could not (S10).
- A local gate, fading-memory gates and a selective gate failed (S6, S8: 0/10 each). The latter two also failed in the grouped layout (S9).
- A 5% labelled nudge toward the stream split discovered 5/10, and 10/10 with slow memory (S10, S15). Those are not label-free.
- Slow memory with the hinge discovered 0/10, failing by key splits (S17). A third hinge term on key identity discovered 6/10 (S19) and 4/10 on new seeds (S22), with 7 of 20 fully routed.

These screens cannot show that a gate routes, for a reason S23 found: a single channel binds the header layout (3/10, and 7/10 with slow memory), so binding there is no evidence of the partition. At eight keys the header task (S28) did not bind even with the perfect gate within 24,000 updates (final accuracy 0.55–0.87, against 0.20–0.24 for one channel). That screen was invalid by its own rule, although it suggests the partition matters there. A distant-cue task that one channel provably cannot solve is still needed.

13 Where the mechanism stands

setting	perfect gate	plain gate	recipe, no restarts	restarts	one channel
$S=2, P=4$, no conv.	10/10	223/400 ^d	HINGE 39/40 ^d ; Muon + WINDOW 80/80 ^d	60/60	0 of > 200
$S=2, P=8$, conv.	6/6	49/70	HINGE 37/40	40/40 ^c	0/24 ^m
$S=4, P=4, k=4$, conv.	10/10	29/80	— ^s	18/40	0/20
$S=4, P=4, k=16$, conv.	6/6	44/80	HINGE 35/40; hinge only 34/40; Muon + WINDOW 30/32	36/40	—
$S=8, P=4, k=16$, conv.	2/2; 6/6 ^c	0/16	HINGE 0/20; with the curriculum 15/36	0/5; 12/36 ^c	—

Table 8: The main Phase III–V configurations, counted over both machines; bound unless marked. The machines ran the same seeds, so these counts are descriptive, not independent samples. ^dDiscovered. ^cWith a curriculum (load at $P = 8$, streams at $S = 8$). ^mOne channel with the same fast-weight memory. ^sNot run on the main line; a ten-seed screen of HINGE under Adam bound 7/10, 5/10 with one channel per stream.

The partition does what one channel cannot. Unchanged from Revision 6: given the partition, every grouped-layout configuration tried binds; one channel with the same memory does not. (In the header layout at eight keys the perfect gate did not bind within its budget; Section 12.)

Discovery is decided in the first few hundred updates, and can be steered. Revision 6 saw outcomes settled early and suspected that they depend on what the memory learns first. Phase V supports both. Slowing the memory for 2400 updates shifts the race toward the stream split (S0,

shown on both machines), and a hinge that fires when the gate’s choice becomes positional delivers a few large, well-timed pushes that remove the position splits. Together they bind without restarts at two streams with four and eight keys and at four streams with sixteen channels; under Muon, where eight keys were not tried, at two streams with four keys and at four streams. The recipe’s work is done by update 2400.

At eight streams the obstacle moves upstream. Before any routing can form, the gate’s recurrent state stops reflecting its input, and in a screen the full loss came when its recurrence became expansive (spectral radius above one). No change to the gate’s readout (hinge, split, optimizer) can route by a stream its state does not carry.

Merges remain at four channels. Spare channels prevent most of them. In screens, a copy of the shared channel’s gate row, aimed by key-position mass, breaks some of the rest; why it fails on the others, even when on target, is open.

14 Retrospective

14.1 Revision 6’s claims

“Multi-channel Hebbian plasticity solves context-conditional binding by partitioning memory” (title). Stands.

“A learned gate finds the partition, reliably with restarts for two streams; four streams are the open problem” (subtitle). Superseded. With the HINGE recipe, two streams, and four with sixteen channels, now bind without restarts in at least 85% of runs on each machine; eight streams are the open problem.

“A simple early check at four streams picked binders with 38/38 precision” (post hoc). Partly confirmed: precise on L (19/19), not on X (17/20, the three exceptions merges).

“Exploratory screens suggest it depends on what the memory learns first.” Supported by a pre-registered test on both machines (S0).

“Restarts are the working recipe.” Replaced at two streams, and at four with sixteen channels, by an early-training recipe.

Pooled post-hoc p -values across X and L. Corrected. Revision 6 pooled the two machines’ discordant pairs and gave p -values (for example $p = 0.002$ for the layout comparison and $p = 0.049$ for sixteen channels at four streams). The two machines ran the same seeds, so these pairs are not independent and the p -values overstate the evidence. Read them as descriptive. No pre-registered verdict depended on them.

14.2 Errors made during Phase V

Replicates treated as independent. Besides Revision 6’s pooled p -values, our first account of the early-recipe test called X and L “independent replicates”. The agent that recorded L’s result corrected the wording before committing it.

“The hinge barely takes part.” After the first Muon screen we read the hinge’s 0–4 firings per run (on 19 of 20 runs) as a sign that Muon with slow memory might not need it. The next screen showed the hinge rescuing 8 of 40 runs. A few firings, at the right moment, were the mechanism.

A merge breaker over-claimed, then mis-specified. We called S24’s row copy a label-free merge breaker before any trigger existed. The trigger we then specified, “one channel holds more than $1.5/k$ of the mass”, would fire on a correctly routed eight-stream gate at $k = 16$ (each channel in use holds $1/8 > 1.5/16$ of the mass). We replaced it with a plateau trigger, but kept an aiming rule, all-position mass, that missed half the time (Section 10).

An underpowered confirmation. We planned the early-recipe test with the screens’ numbers in mind. On fresh seeds the baseline was higher, and our own time rule cut Part B to 12 seeds on X.

Machine differences misattributed. We first attributed X–L differences under Muon to its bf16 kernels alone. X and L never reproduce each other, under Adam either; only X’s two hosts did.

Specifications lost between sessions. The full specification of the slow-start test, and those of three queued screens (S29–S31), never reached the sessions meant to run them. The test was run from a complete replacement specification sent after the session asked for it; the screens were dropped or deferred.

A validity arm without the budget. The eight-key header screen’s perfect gate did not bind within its budget, so the screen gave no answer.

Process. A screening bug let two continuations share optimizer state (S14); it was found and rerun, with no reading changed. Container restarts and two-hour session limits interrupted most long runs; every resumed segment began by reproducing a recorded run bit for bit, and runs in flight restarted from the beginning. On L, X’s short-convolution results file replaced L’s own, and the early-recipe results file first sent from L was a copy of X’s. The test’s pairing checks stopped that run before any training; L’s own file was restored and the test rerun.

15 Methodological findings

Revisions 5 and 6 recorded eleven lessons; Phase V adds five.

Shared seeds make machines dependent. Reusing seeds across machines is right when each machine pairs with its own recorded runs. A fresh-seed confirmation should give each machine its own seeds, so that a result shown on both is two independent confirmations and pooled counts mean something.

Bit-for-bit determinism depends on the processor and its kernels. Adam reproduced across X’s two Xeon hosts, but not between X and L. Muon’s bf16 Newton–Schulz step differed even between the two Xeon hosts. Pair only on one host, and record the kernel path in every results file.

Probe the mechanism before designing the fix. Splits failed at eight streams for a reason no outcome count could reveal. A decoder on the gate’s state and two norms located it in two screens.

Score routing, not binding, wherever the memory has another route. Four streams on four channels and the header layout both let the memory bind without the partition.

Power a confirmation from the baseline it will meet. A screen’s baseline can be low by chance; a time rule should not cut the arms a claim depends on.

16 Limitations

The task. Synthetic, with explicit context tokens; in the grouped layout each key’s stream token is the previous token. Distant cues are unresolved.

The recipe needs task structure. The hinge uses no stream labels, but it needs the key positions and their triple index. A language model has no marked keys, so the analogue (for example, absolute position as the nuisance) is untested.

The model. Three layers, one head, $d = 32$, $N = 256$, sequences of at most 99 tokens, memory half-life about 14 tokens. Every learned-gate Muon run used one learning rate (0.005).

Streams. Two streams, and four with sixteen channels, have a recipe; eight have none.

Replication. The second machine reruns the same seeds: it replicates the procedure, but neither the numerics nor independent samples (Section 15).

Screens. One container, 3 to 40 runs per arm, lenient rules, and several post hoc readings, labelled as such.

17 Distance to a language model

What Phase V adds to the case. Discovery no longer needs per-run restarts at two streams, or at four with sixteen channels: an early phase of 2400 updates suffices, so a single training run can use it. It works under Muon, an optimizer already used to pretrain large language models [4], and on one machine Muon bound four streams sooner.

What stands in the way, roughly in order of risk:

1. *Many contexts.* Eight streams fail. The diagnosis points at the gate’s recurrence, which will need redesign or constraint.
2. *The hinge’s nuisance variables.* They come from the task’s structure. A language model needs a generic version.
3. *Context cues at a distance.* Unresolved; the header layout cannot tell routing from single-channel binding.
4. *Gate compute.* The recurrent gate runs token by token. A contractive or parallel gate, which may also be what eight streams need, would remove that cost.
5. *Memory horizon.* A half-life of about 14 tokens is far below a language model’s context.
6. *Prior art.* Mixture-of-Memories [6] already routes tokens among linear-attention memories at the billion-parameter scale.

A staged path. (i) On CPUs: eight streams (batch 15 first), then a distant-cue task that one channel provably cannot solve. (ii) A GPU port, with Muon and a parallel or contractive gate, on the multi-query associative recall benchmark [7] against linear-attention baselines that include a Mixture-of-Memories router. (iii) Small language models. (iv) Scaling. Stages (i) and (ii) remain cheap and are where the idea is most likely to fail.

18 Conclusion and next steps

Multi-channel Hebbian plasticity in multilayer BDH solves context-conditional binding by partitioning memory, and a label-free gate can now find the partition without restarts at two streams and at four streams with sixteen channels. Two changes do it, both confined to the first 2400 updates: slow the memory, and push the gate away from positional splits when it starts to make them. The obstacle at eight streams is earlier still: the gate’s recurrent state loses the context before any routing forms, and in a screen the full loss came when its recurrence became expansive.

Next:

- Batch 15’s cap screen (running): a cap on the gate’s recurrent gain, alone and with a previous-token input, and the previous-token input without recurrence. If the cap keeps the stream in the gate’s state and binds, a pre-registered eight-stream test follows, with each machine on its own seeds.
- Then the targeted split on the merges that remain, and a distant-cue task that one channel cannot solve.
- Commit L’s results for the stream-recipe, stream-curriculum, slow-start and recipe-scope tests to `results/L/`.
- Then the GPU port and the recall benchmark of Section 17.

A Provenance

test	X: test commit / result commit	L: commit (results file)	runs, X; L
<code>stream_recipe</code>	092937b / 18d5ae1	092937b	83; 90
<code>stream_curriculum</code>	ffdf0aa / 520fe51	631fd62	41; 53
<code>slow_start</code>	9c5939e / b8c6007	9c5939e	110; 110
<code>recipe_scope</code>	a429af9 / c69f1e6	c69f1e6	56; 60
<code>early_recipe</code>	9f25dc8 / 9437e01	9437e01	108; 124

Table 9: Phase V commits on branch `claude/bdh-growth-hebbian-inference-w90069`. The L commit is the one recorded in L’s results file; L’s test code equals X’s test commit. Only L’s `early_recipe` result is in `results/L/` (committed in 00ac0f2); L’s other four are not yet committed. Their verdicts here were regenerated with each test’s own report function from L’s results files, and for `recipe_scope` they match L’s full log. X: Intel Xeon at 2.10 GHz through `stream_recipe`, 2.80 GHz after; L: Intel i7-12650H; PyTorch 2.14.0 throughout.

batch	screens	code commit	runs
2	S4 kWTA warm-up, S5 slow memory, S6 distant cue	6f529b5	83
3	S7 slow memory (new seeds), S8 distant-cue gates, S9 near check	3e70887	70
4	S10 expressibility and nudge, S11 gate reset, S12 position penalty	fd1a46	50
5	S13 hinge, S14 stalled runs, S15 nudge with slow memory	98b2441	59
6	S16 random kick, S17 hinge at a distance, S18 stuck memory	8407985	38
7	S19 key-term hinge, S20 untimed kick, S21 kicks on merges	cea4756	78
8	S22 key-term check, S23 one channel at a distance, S24 row copy	572a00d	78
9	S25 Muon, S26 gate noise, S27 z-loss, S28 eight keys at a distance	d0c31ce	117
10	S32 Muon without the hinge, S33 Muon at four and eight streams	d16ec61	124
11	S34 early window, S35 Muon at sixteen channels	9f47868	102
12	S36 plateau-triggered split	faa1926	30
13	S37 targeted split, S38 gate-state probe	c2b30dc	50
14	S39 gate memory over time, S40 previous-token input	07ccfb7	40
15	S41 gate cap (running), S42 W_h spectrum	1432fe0	14 of 65

Table 10: Exploratory batches on branch `claude/outside-ideas`, container E, one thread per run. Every batch began each segment with a check that reproduced one of X’s recorded runs bit for bit. Batches 2–14 ran on a 2.10 GHz Xeon (two segments of batch 7 excepted), batch 15 on a 2.80 GHz Xeon, where each segment also fingerprinted a fresh Muon reference. Batch 15’s count is at the time of writing: all 10 of S42’s runs and 4 of S41’s 55. S29 and S30 were deferred and S31 dropped; none has run.

References

- [1] A. Kosowski, P. Uznański, J. Chorowski, Z. Stamirowska, M. Bartoszkiewicz. *The Dragon Hatchling: The Missing Link between the Transformer and Models of the Brain*. arXiv:2509.26507, 2025.
- [2] A. Gu, T. Dao. *Mamba: Linear-Time Sequence Modeling with Selective State Spaces*. arXiv:2312.00752, 2023.
- [3] K. Jordan et al. *Muon: An optimizer for hidden layers in neural networks*. Blog post, 2024.
- [4] J. Liu et al. *Muon is Scalable for LLM Training*. arXiv:2502.16982, 2025.
- [5] B. Zoph et al. *ST-MoE: Designing Stable and Transferable Sparse Expert Models*. arXiv:2202.08906, 2022.
- [6] J. Du et al. *MoM: Linear Sequence Modeling with Mixture-of-Memories*. arXiv:2502.13685, 2025.
- [7] S. Arora et al. *Zoology: Measuring and Improving Recall in Efficient Language Models*. arXiv:2312.04927, 2023.